nlmixr² Cheat Sheet

October 2026

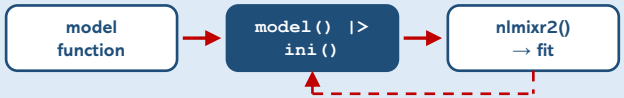
https://nlmixr2.org/

Modifying models with rxode2 & nlmixr2

Change equations with model(), change estimates with ini(), and pipe a fit into either to start the next run from its final estimates.

Writing & piping models

A model is a function with an `ini({})` block (fixed effects `<-`, random-effect variances `~`, residual error) and a `model({})` block (individual parameters, ODEs or `linCmt()`, error model). Every modification returns a new model; `ini` follows the lines automatically.



pipe the fit again: it starts from its final estimates

Template

piping

```
<model_or_fit> |>
  model(<line>, append = <where>,
        auto = <TRUE/FALSE>, cov = <names>) |>
  ini(<par> = <value|fix|unfix|label()>) |>
  nlmixr2(<data>, est = "focei")  # refit
```

Base model in the examples

model

```
one.compartment <- function() {
  ini({
    tka <- 0.45; label("Ka")
    tcl <- 1;    label("Cl")
    tv  <- 3.45; label("V")
    eta.ka ~ 0.6; eta.cl ~ 0.3; eta.v ~ 0.1
    add.sd <- 0.7
  })
  model({
    ka <- exp(tka + eta.ka)
    cl <- exp(tcl + eta.cl)
    v  <- exp(tv + eta.v)
    d/dt(depot)  = -ka * depot
    d/dt(center) = ka * depot - cl / v * center
    cp = center / v
    cp ~ add(add.sd)
  })
}
```

`nlmixr2(one.compartment)` Parse and print without data: a syntax check before fitting.

`fit$finalUi` The model of a fit with its final estimates.

`rxClean()` Clear the cache of compiled models.

model(): lines

Type the line you want. An existing left-hand side is replaced; anything new is inserted where `append` says.

Change a line

`mod |> model(ka <- exp(tka))` Replace the `ka` line; `eta.ka` is dropped from `ini`.

`mod |> model(ka <- tka * exp(eta.ka))` Add BSV back; `eta.ka` gets variance 1 (not mu-referenced).

`mod |> model(cp ~ add(add.err) + prop(prop.err))` Change the error model; new parameters start at 1.

Insert lines: append =

Without `append`, the line with the same left-hand side is replaced.

`append = TRUE` At the end of the model.

`append = FALSE` At the beginning of the model.

`append = d/dt(center)` Directly after that line (any LHS works).

```
mod |> model({
  PD <- 1 - emax * cp / (ec50 + cp)
  effect(0) <- e0
  kin <- e0 * kout
  d/dt(effect) <- kin * PD - kout * effect
}, append = d/dt(center))
```

Remove and rename

`mod |> model(-emax)` Remove the line; parameters only used there (`temax`) leave `ini`.

`mod |> model(emax <- NULL)` Same as `-emax`.

`mod |> rxRename(new = old)` Rename everywhere in `model` and `ini`. Also `dplyr::rename()`.

Combine models

`rxAppendModel(pk |> rxRename(ceff = cp), idr)` Join a PK and a PD model through a shared variable.

`rxAppendModel(pk, idr, common = FALSE)` Join without a shared variable (no warning).

`… |> model(ceff = cp, append = ic50)` Then add the link yourself, after the `ic50` line.

Naming & endpoints

New names are classified by prefix or suffix; promoted parameters start at 1. Anything else becomes a data column (covariate).

Automatic variable detection

`eta ppv psv iiv bsv bpv` Between-subject variability (omega).

`tv t pop err eff cov` Population parameter (theta).

`name in an error model` Residual parameter, e.g. `lnorm.sd`.

`anything else` Covariate; always `wt`, `sex`, `crcl`.

`model(..., cov = "eta.ka")` Force a name to be a covariate.

`model(..., auto = FALSE)` No promotion in this call; promote later with `ini()`.

`options(rxode2.autoVarPiping = FALSE)` Switch detection off for the session.

`rxSetPipingAuto(thetamodelVars = rex::rex("te"))` Only names with `te` become thetas.

`rxSetPipingAuto()` Reset to the default rules.

Multiple endpoints

One prediction can carry several residual models when each is named with `| name`, the endpoint's `cmt`/`dvid` in the data.

two assays, one prediction

```
mod4b <- mod |>
  model(cp ~ add(add.sd) | assay1) |>
  model(cp ~ lnorm(lnorm.sd) | assay2,
        append = TRUE)
```

`mod4b |> model(cp ~ prop(prop.sd) | assay2)` Change one endpoint; without `| assay2` it errors and lists the choices.

`mod4b |> model(-assay2)` Drop one endpoint.

`mod4b$multipleEndpoint` The `cmt`/`dvid` each endpoint uses.

`mod4b$endpointAlias` Internal names (`rx.cp.assay1`).

Two unnamed residual lines on the same left-hand side are rejected: the data could never tell them apart.

nlmixr² Cheat Sheet

October 2026

https://nlmixr2.org/

Modifying models with rxode2 & nlmixr2

Change equations with model(), change estimates with ini(), and pipe a fit into either to start the next run from its final estimates.

Estimates with ini(), piping fits, low-level access, user functions and nlmixr2lib.

ini(): estimates

Values and bounds

`ini(tka = 0.1)` New estimate; `tka <- 0.1` and `tka ~ 0.1` do the same.

`ini(tka <- c(0, 0.1, 0.2))` `c(lower, estimate, upper)`; or just `c(lower, estimate)`.

`ini(tka <- c(-Inf, 0.1, Inf))` Default bounds again (not printed).

`ini(tka = 0.1, tcl = 1, tv = 3)` Several at once; also a `{}` block or a named vector.

`ini(eta.ka + eta.v ~ c(0.6, 0.01, 0.2))` Omega block: variance, covariance, variance.

`ini(~eta)` A named vector prefixed with ~ sets BSV variances.

Fix and unfix

`ini(tka = fix)` Fix at the current value.

`ini(tka = fix(0.1))` Fix at a new value.

`ini(tka = unfix)` Estimate again from the current value.

`ini(tka = unfix(10))` Estimate again from a new value.

Types, labels and order

`ini(~tcl)` Toggle theta ↔ omega (a new omega starts at 1).

`ini(-tcl)` Make it a covariate. Same as `ini(tcl <- NULL)`.

`ini(tcl = 4) / ini(tcl ~ 0.1)` Promote a covariate to a theta / an omega.

`ini(tka = label("Ka (1/h)"))` Label for tables; `label(NULL)` removes it.

`ini(tka = backTransform(exp))` Back-transformation; `NULL` removes it.

`ini(tka = label("Ka"), append = tcl)` Also move `tka` after `tcl`.

Removing covariances

`ini(diag())` All off-diagonal omega elements.

`ini(diag(eta.ka))` Every covariance involving `eta.ka`.

`ini(-cov(eta.ka, eta.v))` Only that pair.

Running & refitting

nlmixr2: piping fits

Everything works on a fit too. The new model starts from the fit's final estimates; refit by piping into `nlmixr2()`.

allometric weight on CL

```
fit2 <- fit |>
  model(cl <- exp(tcl + eta.cl) *
                (WT / 70)^covWtPow) |>
  ini(covWtPow = fix(0.75)) |>
  nlmixr2(theo_sd, est = "focei")
```

`fit |> nlmixr2(data, est = "focei")` Other algorithm, same model. Don't compare OFVs across algorithms.

`fit |> ini(tka = 0.5) |> nlmixr2(est = "posthoc")` Individual estimates only; thetas not re-estimated.

`fit |> model(ka <- exp(tka + bsv.ka)) |> ini(bsv.ka = 0.1)` Add BSV with a start value.

`fit |> model(cp ~ add(add.err) + prop(prop.err))` Additive → combined error (needs DV > 0).

`fit |> ini(tv <- label("Central volume"), append = "tcl")` Relabel and reorder `fit$parFixed`.

`fit$parFixedDf` · `fit$omega` · `fit$objf` Estimates, omega, OFV.

`AIC(fit1, fit2)` · `BIC(fit1, fit2)` Compare models that aren't nested.

`exp(fit$parFixedDf["Sex_V", "Estimate"])` Log-scale effect as a fold change; select rows by name.

Low-level access

For helper functions. These don't keep `ini` and `model` in sync.

`modelExtract(f, endpoint = NA)` All model lines as strings.

`modelExtract(f, cl, lines = TRUE)` One line; `attr(x, "lines")` is its position.

`model(f) <- lines` · `ini(f) <- NULL` Replace the model block; clear `ini`.

`ini <- rxode2(f)$iniDf; ini(f) <- ini` Edit estimates as a data frame.

Extras

User-defined functions

R functions with fixed scalar arguments work inside `model({})`. `rxFun()` converts them to C: multithreaded, with analytic derivatives.

user function

```
gg <- function(x, y) {
  x * y   # no return(): derivatives made
}
g <- function() {
  ini({ tke <- 0.5; add.sd <- sqrt(0.1) })
  model({ ke <- tke
          ipre <- gg(10, exp(-ke * t))
          ipre ~ add(add.sd) })
}
rxClean(); rxFun(gg)
```

`rxFun("fun", c("a","b"), cCode)` Register hand-written C code.

`rxD("fun", list(function(a, b) …))` Derivatives for a C function.

An explicit `return()` gives no derivatives (finite differences instead). No `for`, `lapply` or package functions.

nlmixr2lib

`modeldb` Table of available structural and published models.

`readModelDb("PK_1cmt_des")` Read a model; print it before piping (names like `vc`, `Cc`).

`… |> addTransit(5) |> addEta(c("ka", "cl", "vc"))` Add transit compartments and BSV.

`… |> addResErr(c("addSd", "propSd"))` Set the residual error model.

Gotchas

`start = 1` Auto-promoted parameters start at 1; set real values with `ini()`.

`Sex_V` No recognised prefix: a covariate until `ini(Sex_V = 0.1)`.

`WT / 70` Unknown names are data columns: write reference values as numbers.

`coord_cartesian()` Zoom VPCs with it; `xlim()` drops data first.